

Contact Manager — ATTSW Project Report

Student: Mohamed Jama Farah

Professor: Lorenzo Bettini

University: University of Florence

Course: ATTSW — A.Y. 2025/2026

Exam date: July 20, 2026, 09:30, Aula 005 C.Didat.Morgagni

GitHub: <https://github.com/mohamedjama-farah/Contact-Manager->

1. Introduction

This report describes the Contact Manager application I built for the ATTSW course. The application lets the user add, view, update, and delete contacts, and it stores them in MongoDB.

I wrote the project with Test-Driven Development. I did not write a class or a method before a failing test asked for it, and the git history shows the Red and Green commits for each feature. The project uses unit tests, integration tests, and an end-to-end test. Coverage is kept at 100% for lines and branches with JaCoCo, and the project also uses PIT for mutation testing, SonarCloud for static analysis, Coveralls for the coverage badge, and GitHub Actions for continuous integration.

2. Architecture (MVP)

I used the Model-View-Presenter pattern. The controller only depends on two interfaces, `ContactRepository` and `ContactView`, so I can test it with Mockito mocks and no database or GUI. The real classes, `MongoContactRepository` and `ContactFrame`, are put together only in `ContactManagerApp` when the application starts.

Layer	Class / Interface	Role
Model	<code>Contact</code>	Holds id, name, phone, email; immutable, with equals/hashCode
Repository	<code>ContactRepository</code>	Interface: save, findAll, findById, delete, update
Repository	<code>MongoContactRepository</code>	MongoDB implementation
Controller	<code>ContactController</code>	Business logic between repository and view
View	<code>ContactView</code>	Interface: showAllContacts, showError, contactAdded, contactDeleted, contactUpdated
View	<code>ContactFrame</code>	Swing implementation
App	<code>ContactManagerApp</code>	

Layer	Class / Interface	Role
		main(): reads command-line options and wires the parts together

I excluded `ContactManagerApp` from coverage because its `main()` only reads options and wires the parts, so there is nothing meaningful to unit test in it.

3. Test-Driven Development

Every production class started from a failing test. For each feature I first wrote a test that failed, then wrote the smallest amount of code to make it pass, then cleaned up if there was anything to clean. You can see this in the history, for example RED: `add failing test for updating a contact` followed by GREEN: `implement updateContact`.

4. The model and the id

A `Contact` carries its own id. The id is passed to the constructor, `new Contact(id, name, phone, email)`, and the object is immutable. The repository reads contacts back from MongoDB by building them through this constructor from a stored id field, so the id is never assigned after construction and never taken from MongoDB's internal `_id`.

`equals` and `hashCode` are based on all four fields, which lets the tests compare a whole `Contact` object directly. I test them with `EqualsVerifier`, and I test the constructor, the getters, and `toString` separately.

5. Operations (CRUD)

The application does all four CRUD operations on a contact, and each write operation checks the id first.

- **Create:** `addContact(contact)`. The controller looks the id up with `findById`. If a contact with that id already exists it shows the error "Already existing contact with id ..." and does not save. Otherwise it saves and notifies the view.
- **Read:** `allContacts()` loads all contacts and shows them in the list.
- **Update:** `updateContact(contact)`. The controller looks the id up first; if it is not there it shows "No existing contact with id ...", otherwise it updates the stored record.
- **Delete:** `deleteContact(id)`. The controller looks the id up first; if it is not there it shows an error, otherwise it deletes the record.

6. Tests

The project has three kinds of tests.

6.1 Unit tests (Surefire)

`ContactControllerTest` has 7 tests. It checks the controller with Mockito mocks for the repository and the view: list all, add when the id is new, add when the id already exists (error, no save), update when the id exists, update when the id does not exist (error, no update), delete when the id exists, and delete when the id does not exist (error, no delete).

`ContactTest` has 3 tests: the constructor and getters, `toString`, and `equals/hashCode` with `EqualsVerifier`.

`ContactFrameTest` has 10 tests for the Swing GUI with `AssertJ-Swing`. I create the frame on the Event Dispatch Thread with `GuiActionRunner.execute(...)`, wrap it in a `FrameFixture`, and mock the controller. The tests cover showing contacts, the error dialog, the Add button building a contact and delegating to the controller, the Delete and Update buttons delegating to the controller, the Delete and Update buttons being disabled when nothing is selected, the two buttons becoming enabled once a contact is selected, and the list refreshing after add, delete, and update.

6.2 Integration tests (Failsafe, files ending in IT)

`MongoContactRepositoryIT` has 5 tests. It runs the MongoDB repository against a real MongoDB 6.0 container started by Testcontainers: save and findAll, findById when the contact exists and when it does not, update, and delete. I empty the collection before each test so the tests do not affect each other.

`ContactControllerIT` has 3 tests. It wires the real controller to the real `MongoContactRepository`, with no mocking, and checks that adding, deleting, and updating a contact really change the database.

6.3 End-to-end test (Failsafe, separate execution)

`ContactManagerE2E` has 3 tests and runs the whole application through its `main` file. It starts `ContactManagerApp` with `ApplicationLauncher.application(...).withArgs("--mongo-host=...", "--mongo-port=...").start()`, so it drives the running application the way a user would and does not call the application classes directly. The three tests add a contact, add then delete, and add then update. This test lives in its own `src/e2e` folder and runs in a separate Failsafe execution, apart from the integration tests.

7. Build with Maven

Maven compiles the code, resolves dependencies, and runs the tests.

Surefire runs the unit tests (files ending in `Test`) and skips the `IT` files. The build-helper plugin adds `src/it/java` and `src/e2e/java` as extra test-source folders. Failsafe runs the integration and end-to-end tests in two separate executions: one for the `IT` files and

one for the E2E file. JaCoCo prepares its agent, writes the report, and fails the build if line or branch coverage falls below 100%, with `ContactManagerApp` excluded. PIT runs mutation testing on the controller. SonarCloud and Coveralls send their reports to their dashboards.

7.1 Excluding a SonarQube rule in the POM

`testShowErrorDisplaysDialog()` checks the result with AssertJ-Swing's `requireMessage(...)`, which is a real assertion, but Sonar's rule "Tests should include assertions" (`java:S2699`) does not recognise it. Instead of adding a fake assertion to the test, I excluded that rule for that one test file in `pom.xml` with `sonar.issue.ignore.multicriteria`.

7.2 Enabling the buttons instead of guarding inside the listener

The Delete and Update buttons start disabled. A `ListSelectionListener` on the list turns them on when a contact is selected and off when nothing is selected. Because a disabled button cannot be clicked, the button listeners do not need an `if (index != -1)` check, so there is no half-covered branch left to worry about.

8. Continuous Integration (GitHub Actions)

Every push runs `.github/workflows/build.yml` on `ubuntu-latest`. It checks out the repository with full history, sets up Java 17, and runs `xvfb-run mvn verify coveralls:report -DrepoToken=$COVERALLS_REPO_TOKEN`. `xvfb-run` gives the GUI and end-to-end tests a virtual display, and `verify` runs the whole test pipeline and the coverage check. After that it sets up Java 21 and runs `mvn sonar:sonar` for the SonarCloud analysis. The runners already have Docker, so Testcontainers starts MongoDB with no extra setup.

9. Test summary

Test class	Type	Count	Runner
<code>ContactControllerTest</code>	Unit	7	Surefire
<code>ContactTest</code>	Unit	3	Surefire
<code>ContactFrameTest</code>	Unit (GUI)	10	Surefire
<code>MongoContactRepositoryIT</code>	Integration	5	Failsafe
<code>ContactControllerIT</code>	Integration	3	Failsafe
<code>ContactManagerE2E</code>	End-to-end	3	Failsafe
Total		31	

JaCoCo keeps line and branch coverage at 100%.

10. Source structure

```
contact-manager/  
|-- pom.xml  
|-- .github/workflows/build.yml  
|-- README.md  
|-- src/  
    |-- main/java/com/example/contactmanager/  
    |   |-- app/ContactManagerApp.java  
    |   |-- controller/ContactController.java  
    |   |-- model/Contact.java  
    |   |-- repository/ContactRepository.java  
    |   |-- repository/MongoContactRepository.java  
    |   |-- view/ContactView.java  
    |   |-- view/swing/ContactFrame.java  
    |-- test/java/com/example/contactmanager/  
    |   |-- controller/ContactControllerTest.java  
    |   |-- model/ContactTest.java  
    |   |-- view/swing/ContactFrameTest.java  
    |-- it/java/com/example/contactmanager/  
    |   |-- repository/MongoContactRepositoryIT.java  
    |   |-- controller/ContactControllerIT.java  
    |-- e2e/java/com/example/contactmanager/  
    |   |-- e2e/ContactManagerE2E.java
```

10.1 Repository

<https://github.com/mohamedjama-farah/Contact-Manager->

The git history follows the TDD steps: each feature has its Red and Green commits.

11. Conclusion

The project puts together the techniques from the course: TDD for all the production code, the MVP pattern, three levels of tests, 100% coverage that the build enforces, mutation testing, static analysis, and continuous integration.

Goal	Done
TDD for all production classes	Yes
MVP pattern	Yes
Full CRUD, including Update	Yes
Contact id passed in the constructor and validated on add	Yes
Unit, integration, and end-to-end tests	Yes (31 tests)
End-to-end test through main	Yes

Goal	Done
Integration tests for the repository and the controller	Yes
100% line and branch coverage (JaCoCo)	Yes
Mutation testing (PIT)	Yes
Static analysis (SonarCloud)	Yes
Continuous integration (GitHub Actions)	Yes